Finite-Blocklength Bounds with Prior Optimization: the Φ-view

Nir Elkayam
the `fbl` library

Abstract

`fbl` is a validated library for one-shot / finite-blocklength information theory built on the pairwise-error-probability (PEP) error-spectrum framework. It covers channel coding, rate–distortion (average and excess), and joint source–channel coding (JSCC), in two flavours (exact one-shot / method-of-types). Its distinguishing feature is the *achievability* prior optimization, which we show is a single *convex program* on the simplex, $\max_Q c^\top \Phi(AQ)$, where the kernel of the bound chooses the fixed potential Φ. We solve it directly by a first-order *simplex march* that is exact for any kernel and certifies its own optimality by a water-filling (KKT) condition; the classical exact convex solvers (a quadratic program for the RCU^+ kernel, a bracketing LP otherwise) are special cases retained as validation anchors. We report a self-contained figure suite for four pinned use cases, all numbers backed by a 107-test cross-check suite.

1 Introduction

For each setting the library computes three quantities: the *achievable* (random-coding) bound, the *converse* (meta-converse LP), and—one-shot only—a *Monte-Carlo* realization. The converse optimizes the prior point-wise (a linear program at one threshold). The achievability bound is a kernel integral of the *whole* error spectrum, so its prior optimization is global. We show it is nonetheless a convex program with a common form, and that the optimal prior can be found exactly and certified intrinsically. The practical question the suite answers is then sharp: *given the certified optimal prior, how large is the gap to a memoryless prior?*

2 The unified Φ-view

Fix an output y and a metric $m(x, y) \geq 0$. Under the dithered tie rule the PEP of a candidate drawn from the prior Q_X is uniform on $[0, 1]$, so the value-weighted spectrum and its average against a non-negative kernel κ are

$$F(w \mid y) = \mathbb{E}_{X\sim Q_X}\big[m(X, y)\,\mathbf{1}\{\mathrm{PEP} \leq w\}\big], \qquad J = \int_0^1 F(w \mid y)\,\kappa(w)\,\mathrm{d}w. \tag{1}$$

The prior enters through one linear map—the cumulative-rank profile $(AQ_X)(\nu) = Q_X\{x : m(x, y) \geq \nu\}$—and the value through the functional $c^\top v = \int_0^\infty v(\nu)\,\mathrm{d}\nu$. With the kernel potential $\Phi_\kappa(\sigma) = \int_0^1 \min\{w, \sigma\}\,\kappa(w)\,\mathrm{d}w$ one has the *nonlinear program of the PEP*,

$$\boxed{J(Q_X) = c^\top \Phi_\kappa(AQ_X)} \qquad \text{finite alphabet: } J = \sum_j c_j\,\Phi_\kappa(\sigma_j), \tag{2}$$

with σ_j the cumulative masses (sorted by metric) and c_j the value-gaps. The potential satisfies $\Phi_\kappa(0) = 0$, $\Phi'_\kappa(\sigma) = \int_\sigma^1 \kappa \geq 0$, $\Phi''_\kappa = -\kappa$, so Φ_κ is concave; since AQ_X is linear and $c \geq 0$ in the matched case, J is **concave in** Q_X and prior optimization is a convex program over the simplex. Two tails occur: the *error* tail (channel/JSCC) maximizes a concave J; the *correct* tail (rate–distortion) minimizes a convex J in the dual (CDF) variable.

The kernel chooses the program. Table 1 lists the potentials. The converse is the point-mass kernel $\kappa = \delta_{w_0}$, whose ramp potential is piecewise-linear (an LP). The RCU$^+$ potential is a clamped parabola—*quadratic*—so its convex program is a quadratic program (QP). The exact-RC and rate–distortion potentials are non-polynomial, handled either by a bracketing LP (secant/tangent of Φ) or exactly by the march below.

bound	potential Φ_κ	shape	program
converse, level w_0	$\min\{\sigma, w_0\}$	piecewise-linear	LP
channel, RCU$^+$	$\sigma - \frac{M-1}{2}\sigma^2$ (clamped)	quadratic	QP
channel, exact	$\frac{1}{M}\big(1-(1-\sigma)^M\big)$	concave	bracket / march
rate–distortion, exact	$(1-\tau)^M$	convex	bracket / march
rate–distortion, smooth	$e^{-M\tau}$	convex	bracket / march

Table 1: The potentials instantiating (2) (M the codebook size). The QP is simply the case of a *quadratic* Φ.

The simplex march and its certificate. We maximize $f = \text{sense}\cdot J$ (sense $= +1$ channel/JSCC, -1 RD) directly on the simplex by projected gradient / Frank–Wolfe, using the analytic water-fill gradient

$$\nabla f(Q) = A^\top\big(c \odot \Phi'(AQ)\big), \qquad g(x) = \sum_y \sum_{i \text{ fed by } x} \text{ratio}_i \sum_{j \geq i} c_j\, \Phi'(\sigma_j). \tag{3}$$

The prior obeys a *product* of simplices (one global simplex for channel/RD; one per source-type block for JSCC). Optimality is the **water-filling KKT condition**—∇f flat on the support and dominated off it, per block—and the Frank–Wolfe gap $g(Q) = \sum_b \big(\max_b \nabla f - \langle \nabla f_b, Q_b\rangle\big)$ is a rigorous suboptimality certificate, $0 \leq f(Q^\star) - f(Q) \leq g(Q)$. Thus the achieved bound is always valid (a feasible prior), and $g(Q)$ brackets the prior-optimized optimum without any second solver. The march is exact for every kernel; the QP/bracketing-LP are cvxpy instantiations of the *same* program, kept as independent machine-precision anchors.

3 The library

The code is a small dependency DAG (infra → engines → prior-opt → examples). The prior-opt layer is `phi_view` (the relaxation $J = c^\top \Phi(AQ)$, its potentials and evaluators) and `phi_simplex` (the march and the `check_kkt` certificate); the exact convex solvers remain as anchors.

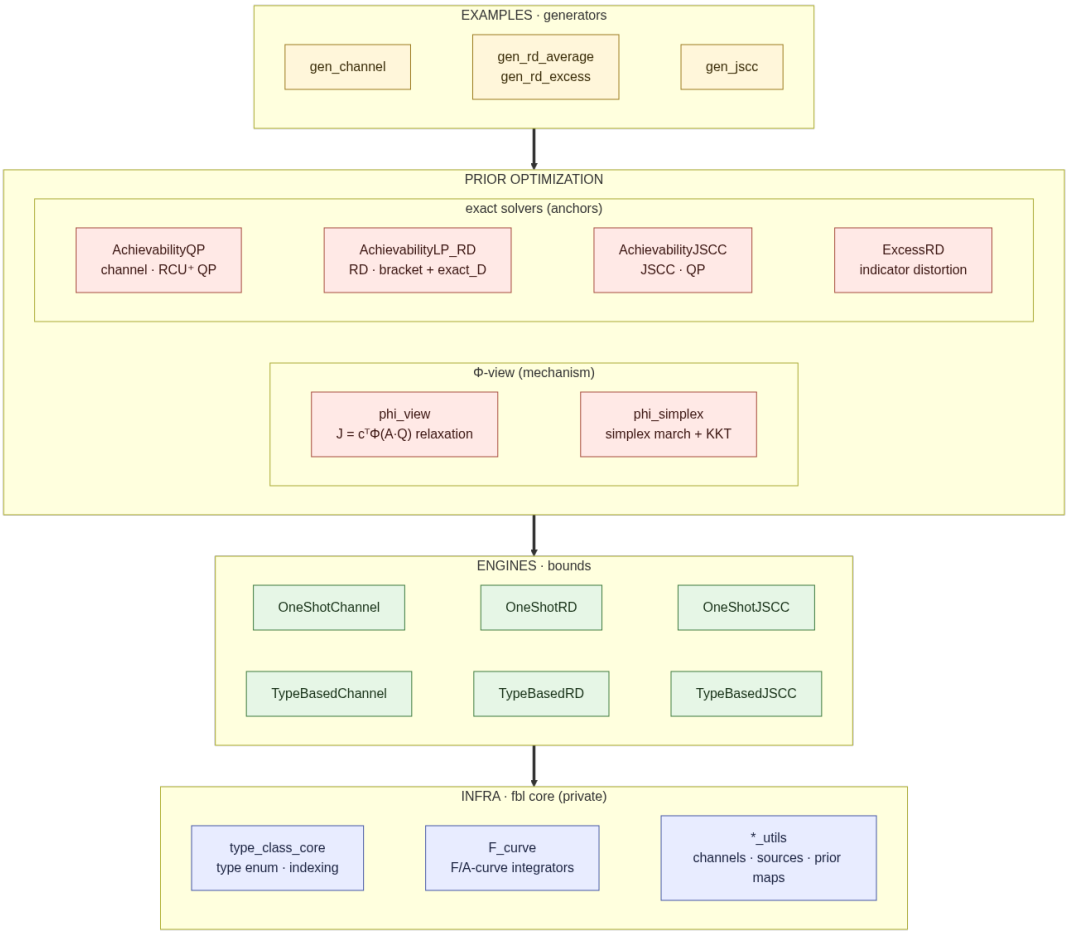


Figure 1: Module dependency DAG. Prior optimization is the Φ-view mechanism (`phi_view`/`phi_simplex`) with the exact solvers as anchors.

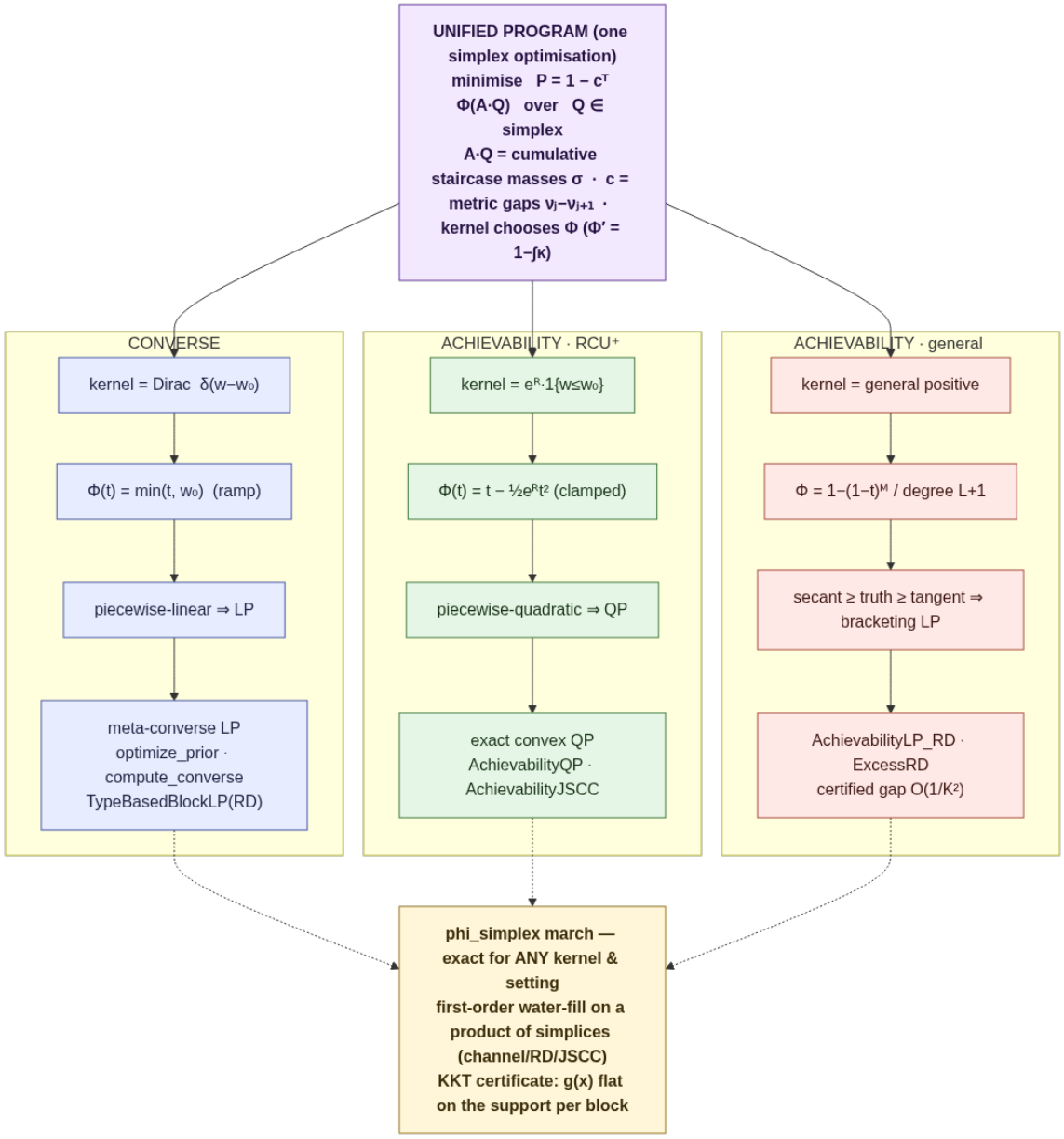


Figure 2: The unified program: the kernel selects Φ (Dirac$\to$LP, RCU$^+$ $\to$QP, general$\to$bracket), and the simplex march solves all of them exactly.

Validation. No closed-form oracle exists, so every quantity is cross-checked (107 tests): one-shot $\leftrightarrow$ type-based at small n; RCU expectation $\leftrightarrow$ Monte-Carlo; converse $\leq$ achievable; the Φ-view identity $J = c^\top \Phi(AQ)$ against an independent computation in both representations and all settings; and the march optimum against the exact solvers *and* the intrinsic KKT certificate (rejected off the optimum).

4 Results

Each use case is pinned to a single channel/source. **G1** validates the bound against Monte-Carlo at small n; **G2** is the prior gap (optimal achievable prior vs. optimal memoryless vs. the two marginal-memoryless priors); **G3** compares exact random coding to a closed-form surrogate; **G4** contrasts the error/distortion spectra of the converse- and achievability-optimal priors. The achievability-optimal prior is the KKT-certified march; the bound is the exact random-coding kernel. G1 stays at $n = 8$; G2–G4 are shown at $n = 8$ and $n = 20$.

4.1 Channel coding — $Z(0.1)$

The non-product prior gain peaks at low rate and grows with n ($\approx 2.0\%$ at $n = 8$, $\approx 3.3\%$ at $n = 20$). The converse-optimal prior, reused for achievability, is $5\times$ worse at $n = 8$ and $15\times$ at $n = 20$.

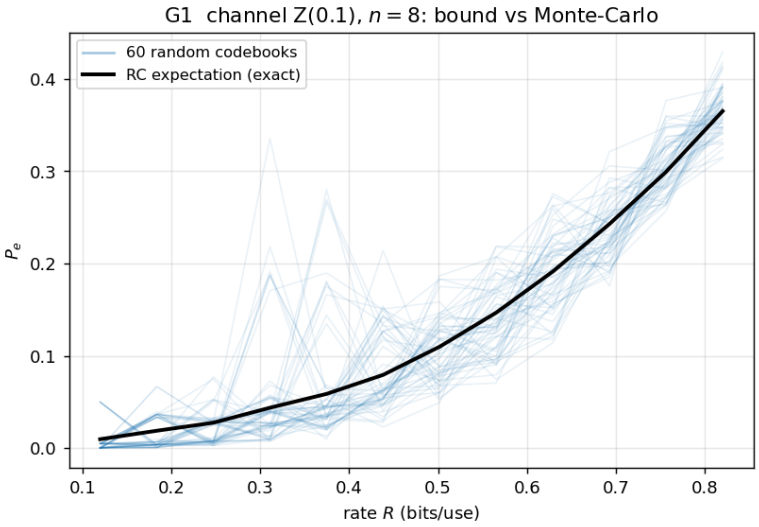
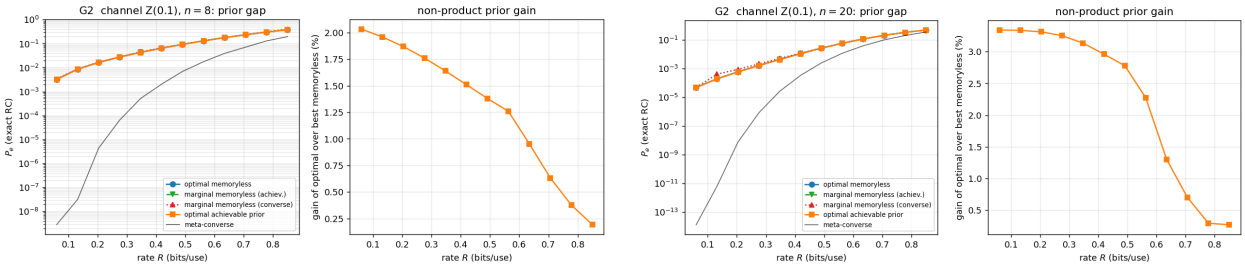
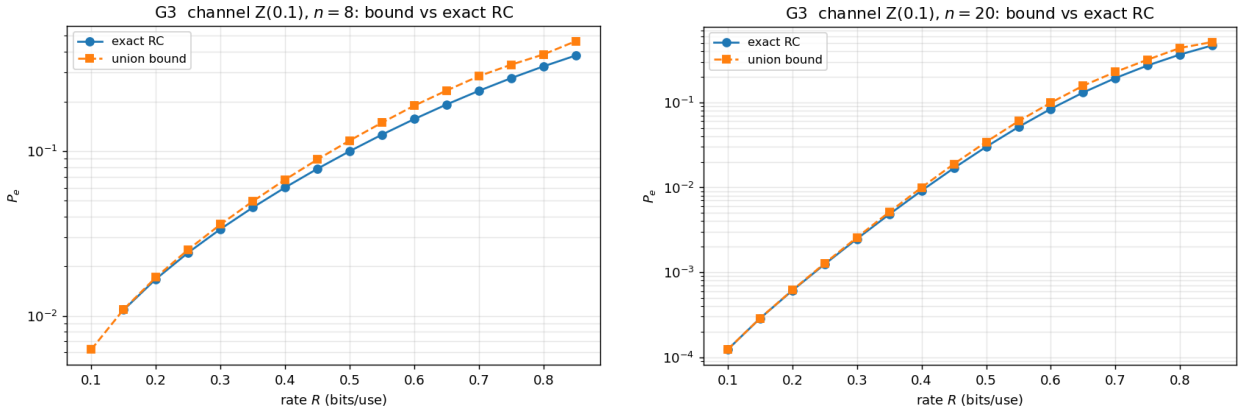


Figure 3: G1 ($n = 8$): random-coding expectation vs. 60 Monte-Carlo codebooks.



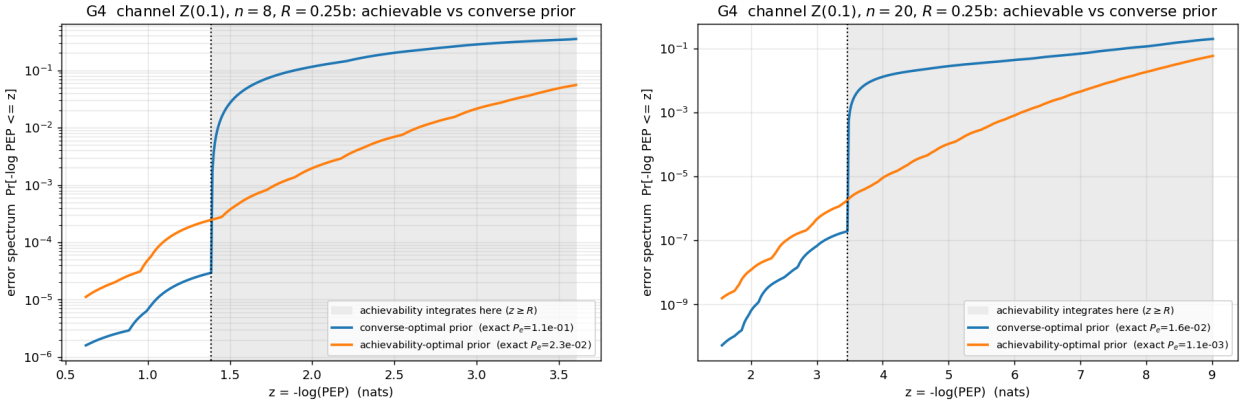
(a) $n = 8$ (b) $n = 20$

Figure 4: G2 — the prior gap (exact kernel). Right panels: gain of the optimal prior over the best memoryless prior.



(a) $n = 8$ (b) $n = 20$

Figure 5: G3 — exact random coding vs. the union bound.



(a) $n = 8$ (b) $n = 20$

Figure 6: G4 — error spectrum of the achievability- vs. converse-optimal prior.

4.2 Rate–distortion, average — BMS(0.25) + Hamming

The gain over the best memoryless prior is $\approx 5.4\%$ ($n = 8$) / 5.9% ($n = 20$); the converse and achievability priors are nearly identical ($D : 0.1551 \to 0.1529$ at $n = 8$).

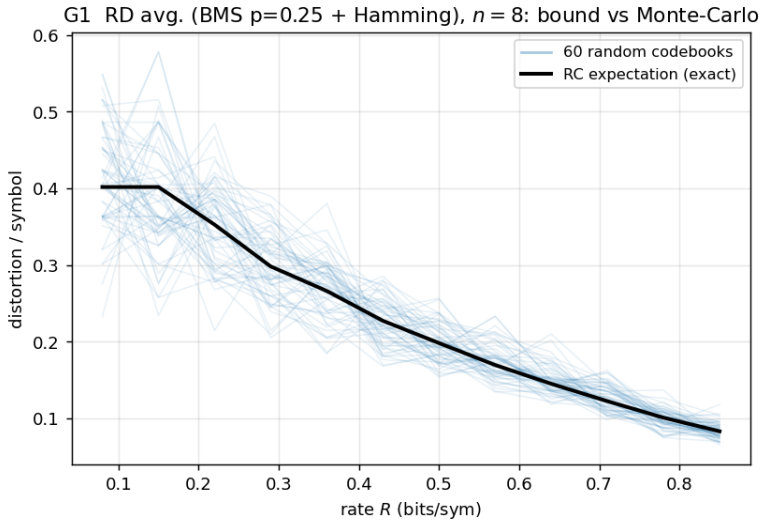
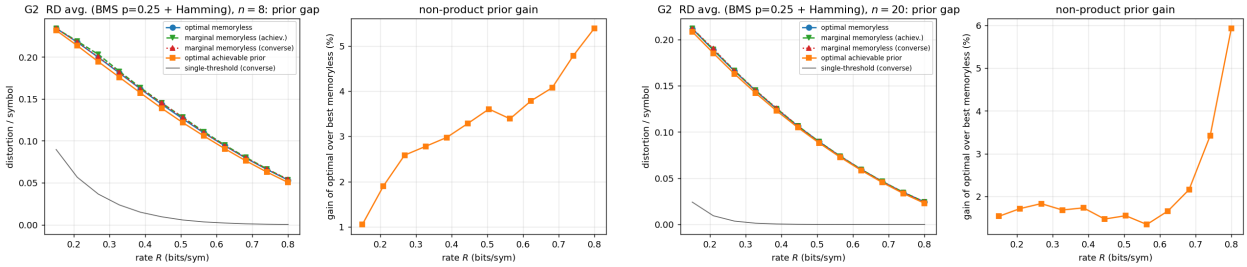
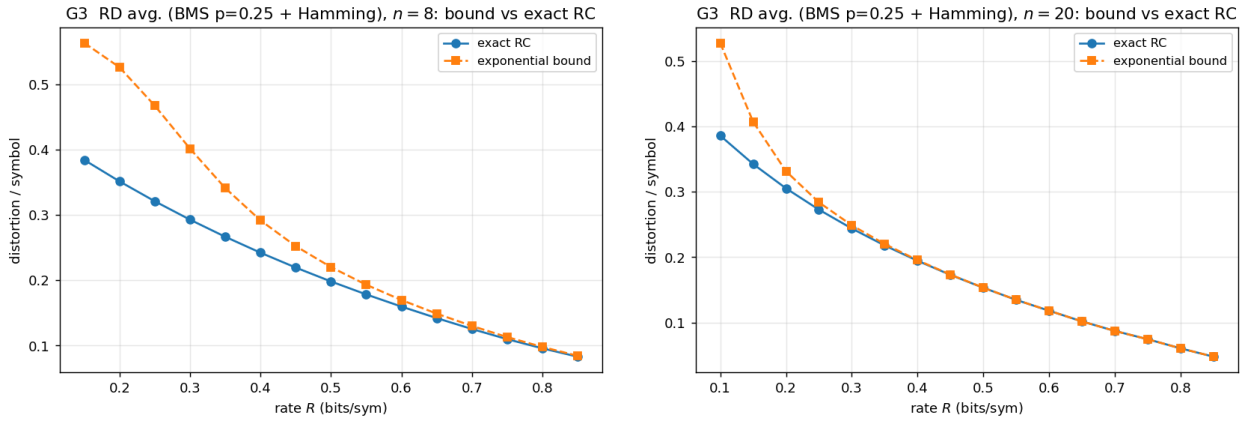


Figure 7: G1 ($n = 8$): realized distortion vs. the random-coding expectation.



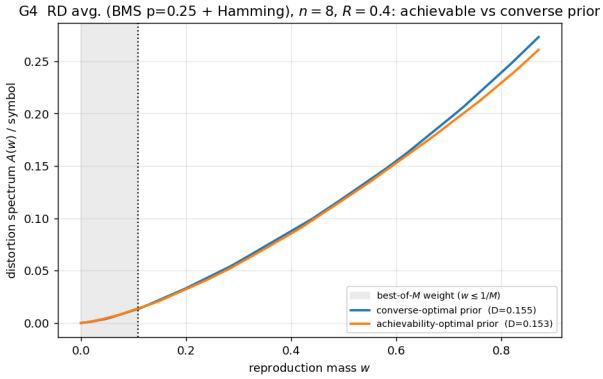
(a) $n = 8$ (b) $n = 20$

Figure 8: G2 — prior gap (exact best-of-M kernel).

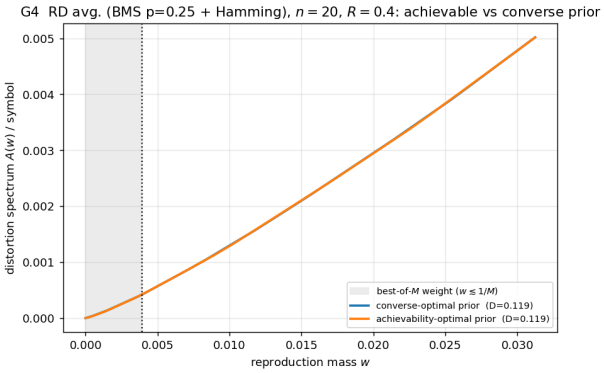


(a) $n = 8$ (b) $n = 20$

Figure 9: G3 — exact random coding vs. the exponential surrogate.



(a) $n = 8$

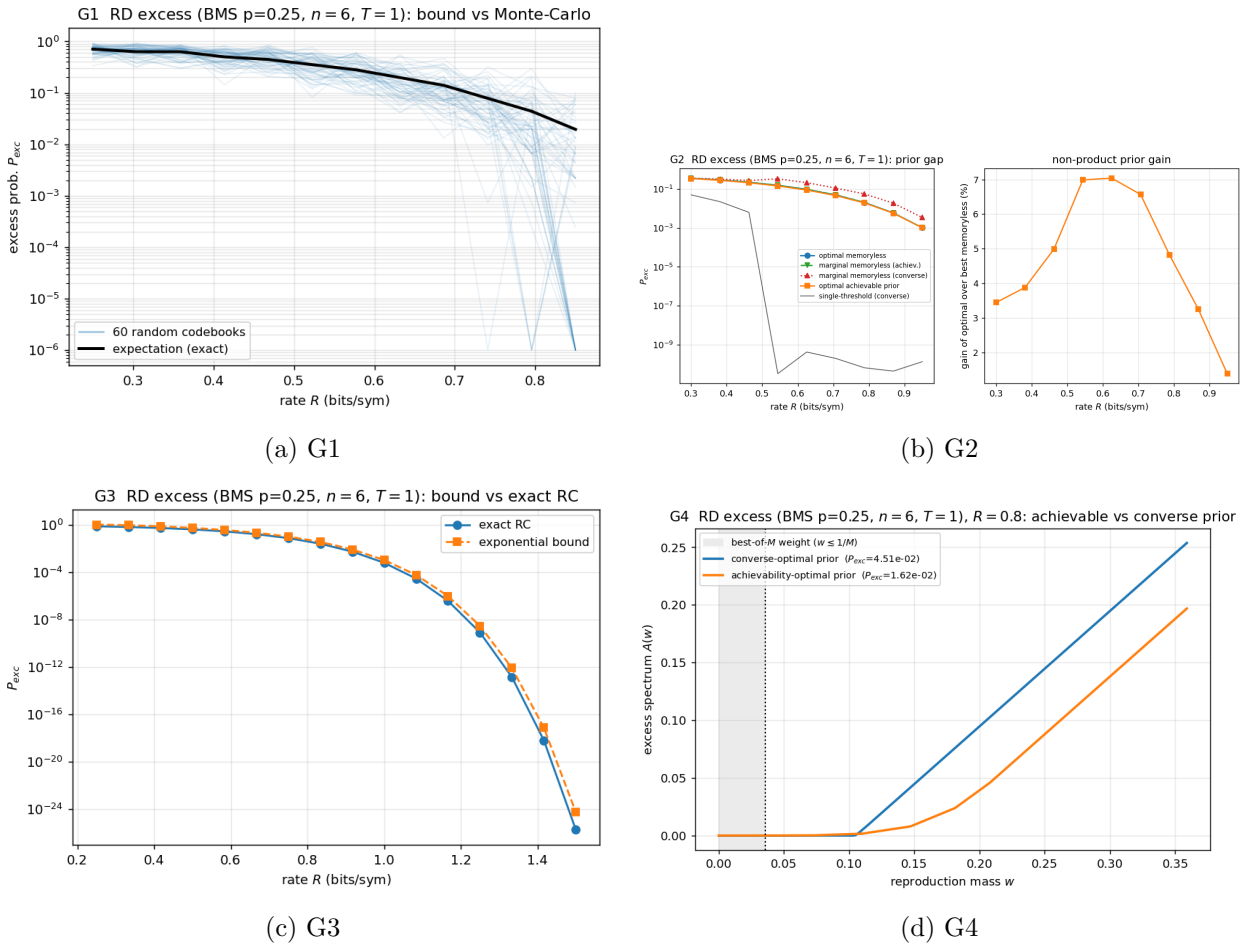


(b) $n = 20$

Figure 10: G4 — distortion spectra (nearly identical priors).

4.3 Rate–distortion, excess — BMS(0.25), $T = 1$, $n = 6$

Excess distortion is the best-of-M of the block indicator, a lifted quantity, so $n = 6$. The gain peaks at $\approx 7\%$; reusing the converse prior costs $2.8\times$ (P_{exc} : 4.5e−2 vs. 1.6e−2).



(a) G1 (b) G2

(c) G3 (d) G4

Figure 11: Excess distortion: bound vs. MC (G1), prior gap (G2), exact vs. bound (G3), and the excess spectra (G4); the converse-marginal prior is the worst here.

4.4 JSCC — BMS(0.07) over BSC(0.05), $H < C$

With $H = 0.366 < C = 0.714$ bit the source is transmissible. The coded bounds (converse, and the RCU$^+$ achievable bound at the march optimum) fall with n, while the uncoded symbol-by-symbol scheme degrades; they cross near $n \approx 4$.

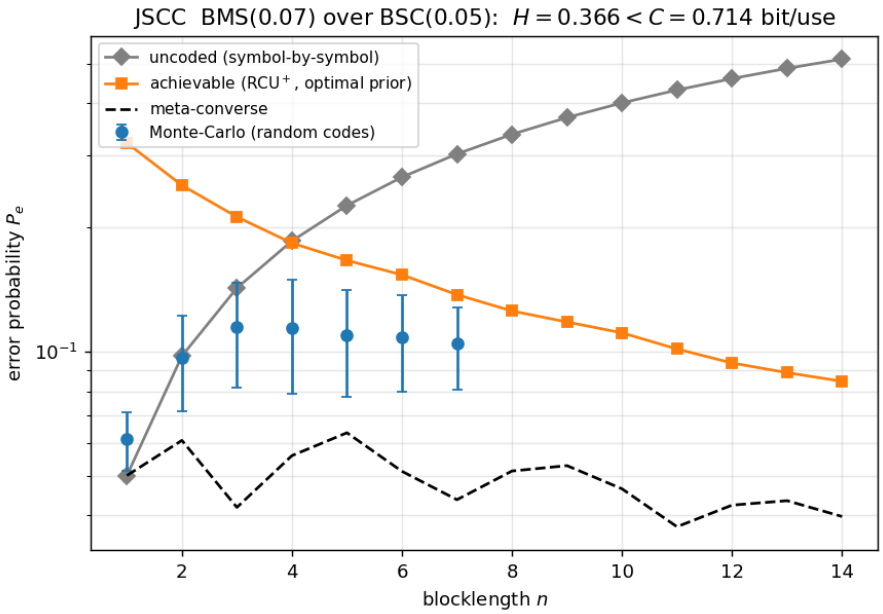


Figure 12: JSCC error probability vs. blocklength: meta-converse, achievable (RCU$^+$ via the simplex march), Monte-Carlo of random codes, and the uncoded baseline.

5 Conclusion

Prior optimization of the achievability bound is one convex program $\max_Q c^\top \Phi(AQ)$; the kernel chooses Φ, and a single first-order simplex march—KKT-certified, exact for any kernel—solves it across channel coding, rate–distortion, and JSCC. With the certified optimum in hand the headline finding is rigorous rather than heuristic: for these minimal settings the memoryless prior is within a few percent of optimal, while the *converse* prior, reused for achievability, can be far from it.